Part A: Object-Oriented Design



Abu Saleh Musa Miah (PhD)

Presentation Outline

- ✓ Object-Oriented Analysis and Design (OOAD)
- ✓ History
- ✓ Scenario-1: DUMS
- ✓ Scenario-2: Agricultural Automation System (AAS)
- ✓ Conceptual Design
- ✓ Technical Design
- ✓ OOAD Thinking
- ✓ CRC:Class, Responsibility and Collaborator

Object-Oriented Analysis and Design (OOAD)

- ✓ A method for analyzing and designing a system by thinking in terms of objects (real-world entities) rather than just functions or logic.
- ✓ It helps us move from requirements (what the system should do) to a design (how the system should be built).
- ✓ Object-Oriented Analysis and Design is about planning before coding.
- ✓ It has two parts:
  - ➢ Conceptual Design (what to build)
  - ➢ Technical Design (how to build it)
- ✓ Using object-oriented principles and tools like CRC cards ensures your software will be structured, maintainable, and reusable.





Why?

- ✓ Part of the foundation for becoming an experienced software architect.
- ✓ Upon completion of this course, we will be able to:
  - ➢ Explain object-oriented analysis and design.
  - ➢ Engage in object-oriented modelling.
  - ➢ Explain design principles for object-oriented programming





Why People First Used Object-Oriented Programming

- ✓ **Software was getting too complex**
- ✓ In the 1960s, programs were written mostly in **procedural languages** (like Fortran, COBOL, C).
- ✓ These worked fine for small programs, but as projects grew into *hundreds of thousands of lines*, it became very hard to maintain.
- ✓ Problems:
  - ➢ Data and functions were separated (functions acted on global data → messy).
  - ➢ Hard to reuse code (you had to copy-paste functions).
  - ➢ Difficult to model *real-world entities*.





OOP solved these problems

- ✓ OOP grouped **data + behavior** together inside "objects".
- ✓ Objects could represent **real-world things** (e.g., a *Car* object has data like color, speed and functions like accelerate(), brake()).
- ✓ Reusability: You could reuse and extend classes without rewriting everything.
- ✓ Abstraction: You could hide details and show only what mattered.
- ✓ This made large systems easier to design, understand, and maintain.





Who Invented OOP? (History)

- ✓ **1960s – Simula 67 (Norway):** First OOP language, simulations (like simulating ships, traffic, factories). Classes, objects, and inheritance → the foundations of OOP.

- ✓ **1970s – Smalltalk (USA):** popularized the term *Object-Oriented Programming*.
- ✓ fully OOP (everything is an object)., Influenced GUI development and later languages.
- ✓ **1980s – C++ (Denmark): Bjarne Stroustrup** extended C with classes → **C++.**, OOP into mainstream software engineering.
- ✓ **1990s – Java (USA):** Sun Microsystems introduced **Java** (by James Gosling). Platform-independent, heavily OOP, designed for the internet era. Became very popular for enterprise software.

- ✓ Reason OOP was used first: To manage software complexity by modeling real-world concepts as "objects".
- ✓ Inventors:Simula 67 → Dahl & Nygaard (first OOP language).
- ✓ Smalltalk → Alan Kay (spread OOP philosophy).
- ✓ C++ & Java → made OOP widely practical.





System Scenario-1: "Dhaka Urban Management System (DUMS)"

- ✓ Dhaka has 3 major problems:
  - ➢ House allocation (overpopulation, unequal distribution)
  - ➢ Garbage management (waste disposal everywhere)
    https://www.youtube.com/watch?v=TcWPIXDQhz8
  - ➢ Traffic control (congestion, poor flow)
    https://www.youtube.com/watch?v=b_mCu2_RQg4
- ✓ We will design a software system to monitor and manage these problems.





System Overview

"Dhaka Urban Management System (DUMS)" is a centralized software platform designed to help city authorities manage three major urban problems in Dhaka: **housing allocation, garbage management, and traffic control**. Citizens, government officials, garbage workers, and traffic authorities will use the system through a web or mobile application.

The system will collect real-time data, store it in a central database, and use AI/ML-based decision support to improve housing distribution, waste collection, and traffic flow.





System Scenario-1: "Dhaka Urban Management System (DUMS)"

- ✓ Goal: A centralized platform to manage housing allocation, garbage collection, and traffic control in Dhaka.
- ✓ Housing Allocation Module
  - ➢ Citizens apply for housing permits.
  - ➢ Government reviews applications based on availability.
  - ➢ Automated allocation based on income, family size, and location /area.
- ✓ **Garbage Management Module**
  - ➢ Track garbage collection zones.
  - ➢ Assign cleanup schedules.
  - ➢ Citizen complaints for uncollected waste.
- ✓ **Traffic Control Module**
  - ➢ Real-time traffic monitoring and Dynamic signal adjustments.
  - ➢ Accident/blockage reporting.

Main Actors

- ✓ Step 1: Understanding the problem domain
- ✓ We identify the main objects (entities) from the problem

Actor	Role
Citizen	Applies for housing, reports garbage issues, reports traffic accidents/blockages
Housing Authority	Reviews housing applications and approves allocation
City Authority/Admin	Monitors the whole system and makes policy decisions
Garbage Management Officer	Creates collection schedules and assigns trucks
Garbage Truck Driver	Collects waste from assigned zones
Traffic Control Officer	Monitors congestion and handles accidents/blockages
AI/ML System	Suggests housing allocation, garbage routes, and traffic signal timing
External Systems	Dhaka City Corporation, BRTA, traffic sensors, GPS systems





Use Case 1: Housing Allocation

Use Case Name: Apply and Allocate Housing Permit

Goal : To allocate available housing fairly based on income, family size, and preferred location.

Primary Actor : Citizen

Supporting Actors: Housing Authority, AI/ML Allocation System, City Authority System

Preconditions: The citizen must be registered in the DUMS platform. Housing data must be available in the system database.





Main Scenario

- The citizen logs into the DUMS mobile or web app.
- The citizen selects “Apply for Housing Permit.”
- The citizen submits personal information, income level, family size, current address, and preferred area.
- The system verifies the application and stores it in the database.
- The AI/ML component analyzes housing availability, applicant income, family size, and location demand.
- The system generates a ranked allocation list.
- The Housing Authority reviews the recommendation.
- If the application is valid, the authority approves the allocation.
- The citizen receives a notification with housing permit details.
- The database is updated with the new allocation record.

Alternative Scenario: If no house is available in the preferred area, the system suggests nearby alternative areas.

Output: A fair and automated housing allocation decision.



Use Case 2: Garbage Management

- **Use Case Name: Report and Manage Uncollected Garbage**
- **Goal:** To ensure timely garbage collection and reduce waste disposal problems in Dhaka.
- **Primary Actor:** Citizen
- **Supporting Actors:** Garbage Management Officer, Garbage Truck Driver, Dhaka City Corporation API
- **Preconditions:** Garbage zones, bins, trucks, and collection schedules must be registered in the system.
- **Main Scenario**
  - ✓ A citizen notices uncollected garbage in their area.
  - ✓ The citizen opens the DUMS app and selects "Report Garbage Issue."
  - ✓ The citizen uploads a photo, location, and short description.
  - ✓ The system identifies the garbage zone using GPS/location data.
  - ✓ The complaint is stored in the database and sent to the Garbage Management Officer.
  - ✓ The system checks the nearest available garbage truck.
  - ✓ The AI/ML routing system suggests the most efficient truck route.
  - ✓ The Garbage Management Officer assigns the task to a truck driver.
  - ✓ The truck driver collects the waste and updates the task status.
  - ✓ The citizen receives a notification that the complaint has been resolved.
- **Alternative Scenario:** If no truck is available, the complaint is placed in the next available collection schedule.
- **Output:** Cleaner city zones and faster response to waste complaints.

Use Case 3: Traffic Control

- **Use Case Name: Monitor Traffic and Adjust Signals Dynamically**
- **Goal:** To reduce traffic congestion and improve vehicle flow in Dhaka.
- **Primary Actor:** Traffic Control Officer
- **Supporting Actors:** Vehicle, Traffic Signal, Citizen, BRTA API, AI Traffic System
- **Preconditions:** Traffic signals, road sensors, cameras, and vehicle flow data must be connected to the system.
- **Main Scenario**
  - ✓ The traffic monitoring system collects real-time data from road sensors, cameras, and GPS-enabled vehicles.
  - ✓ The system detects heavy congestion at a major intersection.
  - ✓ The AI/ML traffic module analyzes vehicle density and road flow.
  - ✓ The system automatically adjusts traffic signal timing.
  - ✓ Green-light duration is increased for the congested road.
  - ✓ The Traffic Control Officer monitors the result through the central dashboard.
  - ✓ If congestion decreases, the signal timing returns to normal.
  - ✓ The system stores traffic flow data for future analysis.

Alternative Scenario: If a citizen reports an accident or road blockage, the system alerts the Traffic Control Officer and suggests alternative routes.

Output: Improved traffic flow, reduced congestion, and faster accident response.

Integrated System Scenario

❖ **Scenario Title: A Day in Dhaka Managed by DUMS**

✓ In the morning, a citizen named Rahim logs into the DUMS app and applies for a housing permit because his family is living in an overcrowded area. The system collects his income, family size, and preferred location. The AI-based housing module analyzes available houses and suggests a suitable allocation. The Housing Authority reviews and approves the application.

✓ Later, Rahim notices a large amount of garbage near his neighborhood. He reports it through the same app by uploading a photo and location. The garbage module identifies the zone, assigns the nearest garbage truck, and updates the complaint status after collection.

✓ During the evening rush hour, traffic congestion increases near a major road intersection. The traffic control module detects the congestion using real-time monitoring data. The system dynamically changes the traffic signal timing to improve vehicle flow. At the same time, if an accident is reported, the system alerts traffic officers and suggests alternative routes.

✓ Through one centralized platform, DUMS helps the city authority make faster, fairer, and data-driven decisions for housing, garbage, and traffic management.





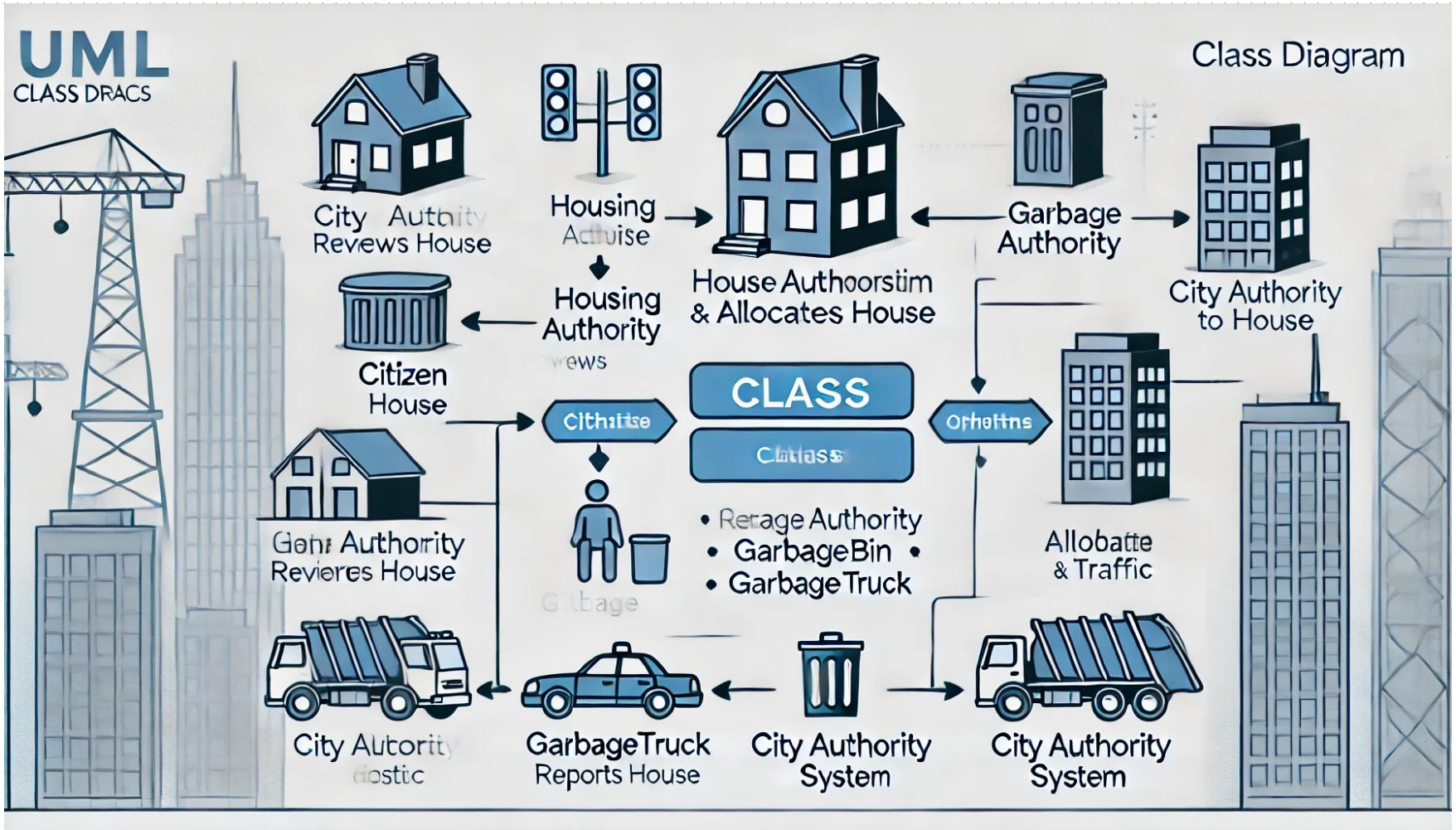
Summary of Use Cases

Module	Main Use Case	Main Benefit
Housing Allocation	Apply and allocate housing permits	Fair distribution of housing
Garbage Management	Report and collect uncollected waste	Cleaner city environment
Traffic Control	Monitor traffic and adjust signals	Reduced congestion
Central Dashboard	Monitor all city services	Better urban decision-making

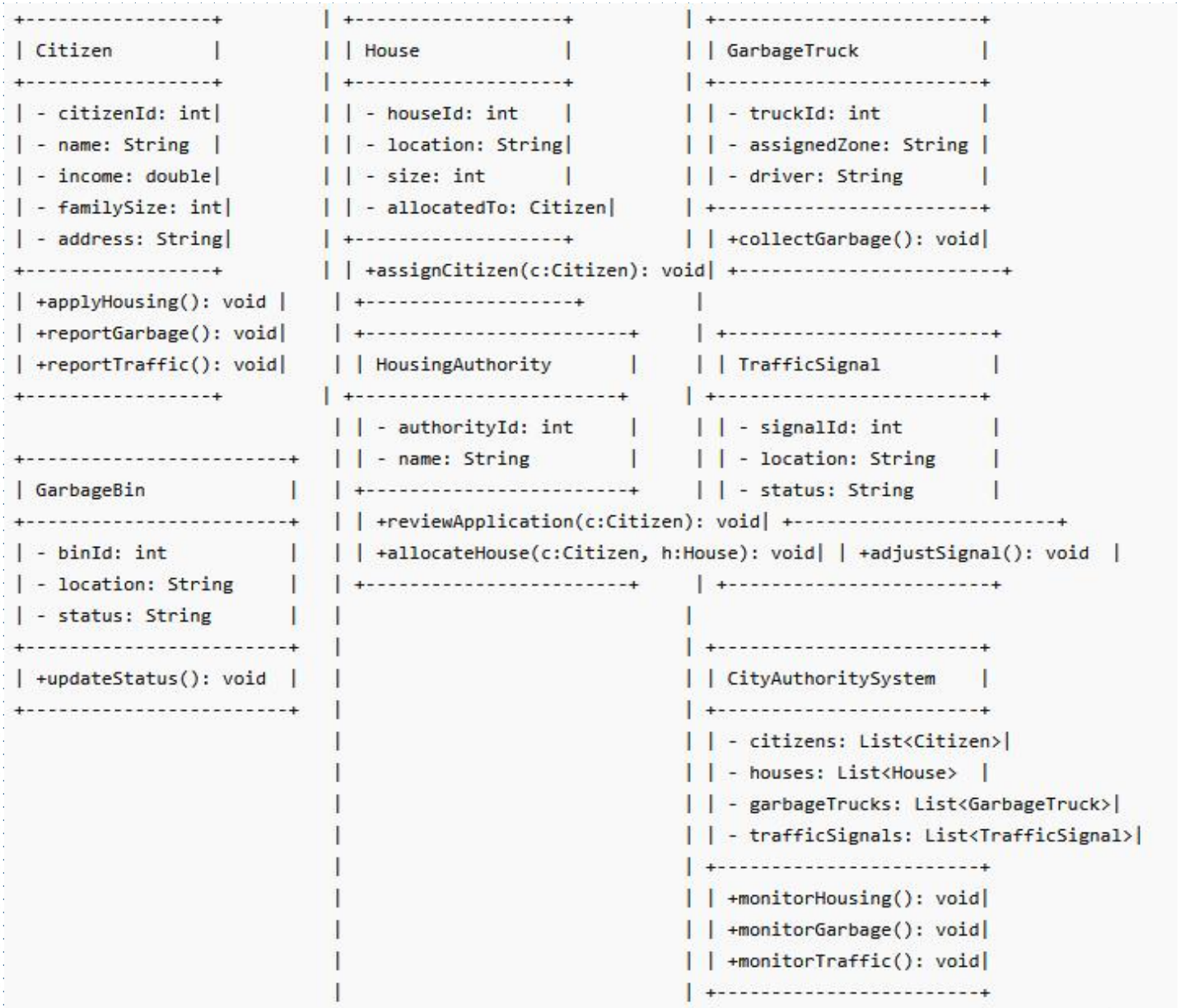
Final Use Case Statement

The "Dhaka Urban Management System" allows citizens and government authorities to work through one centralized digital platform. Citizens can apply for housing, report garbage problems, and submit traffic complaints, while city authorities can monitor real-time data, assign resources, and make AI-supported decisions. This system helps Dhaka manage overpopulation, waste disposal, and traffic congestion in a smarter and more organized way.

UML Diagram


UML
CLASS DRACS
Class Diagram
City Authority
Reviews House
Housing
Housing
Authority
House Authoorstim
& Allocates House
Garbage
Authority
City Authority
to House
Citizen
House
CLASS
Garbage Authority
Revieres House
• GarbageBin •
• GarbageTruck
Allobatte
& Traffic
City Autorit
GarbageTruck
Reports House
City Authority
System
City Authority
System

UML Diagram


Citizen
- citizenId: int
- name: String
- income: double
- familySize: int
- address: String
+applyHousing(): void
+reportGarbage(): void
+reportTraffic(): void
GarbageBin
- binId: int
- location: String
- status: String
+updateStatus(): void
House
- houseId: int
- location: String
- size: int
- allocatedTo: Citizen
+assignCitizen(c:Citizen): void
HousingAuthority
- authorityId: int
- name: String
+reviewApplication(c:Citizen): void
+allocateHouse(c:Citizen, h:House): void
GarbageTruck
- truckId: int
- assignedZone: String
- driver: String
+collectGarbage(): void
TrafficSignal
- signalId: int
- location: String
- status: String
+adjustSignal(): void
CityAuthoritySystem
- citizens: List<Citizen>
- houses: List<House>
- garbageTrucks: List<GarbageTruck>
- trafficSignals: List<TrafficSignal>
+monitorHousing(): void
+monitorGarbage(): void
+monitorTraffic(): void

Sequential Model (Example for Housing Allocation)

Scenario: Citizen applies for housing → AI system suggests allocation → Housing Authority approves → Citizen notified.

Steps (Sequence Diagram in Text):

```
Citizen -> DUMS_UI: Submit Housing Application
DUMS_UI -> Database: Store Application Data
Database -> AI_Module: Analyze eligibility & generate
allocation suggestion
AI_Module -> HousingAuthority: Send suggested allocation
HousingAuthority -> AI_Module: Approve or adjust suggestion
HousingAuthority -> Database: Update house allocation
Database -> DUMS_UI: Notify citizen of allocation
DUMS_UI -> Citizen: Display allocated house
```

Similarly, garbage collection and traffic control sequences follow:

- **Garbage:** Citizen reports → System identifies zone → Assign truck → Truck collects → Citizen notified
- **Traffic:** System monitors sensors → Detect congestion → Adjust traffic signal → Notify officer → Log for analytics





Java Code Skeleton (Basic OOP Implementation)

Simplified Java version to illustrate the core functionality:

```
class Citizen {
    int citizenId;
    String name;
    double income;
    int familySize;
    String address;

    public Citizen(int id, String name, double income, int familySize, String address) {
        this.citizenId = id;
        this.name = name;
        this.income = income;
        this.familySize = familySize;
        this.address = address;
    }

    public void applyHousing(HousingAuthority ha, CityAuthoritySystem city) {
        System.out.println(name + " applied for housing.");
        ha.reviewApplication(this, city);
    }

    public void reportGarbage(String location) {
        System.out.println(name + " reported garbage at: " + location);
    }
```

```
class House {
    int houseId;
    String location;
    int size;
    Citizen allocatedTo;

    public House(int id, String loc, int size) {
        this.houseId = id;
        this.location = loc;
        this.size = size;
    }

    public void assignCitizen(Citizen c) {
        this.allocatedTo = c;
        System.out.println("House " + houseId + " assigned to " + c.name);
    }
```

```
class GarbageTruck { (if added)
    int truckId;
    String assignedZone;
    String driver;
```

Java Code Skeleton (Basic OOP Implementation)

Simplified Java version to illustrate the core functionality:

```
class HousingAuthority {
    int authorityId;
    String name;

    public HousingAuthority(int id, String name) {
        this.authorityId = id;
        this.name = name;
    }

    public void reviewApplication(Citizen c, CityAuthoritySystem city) {
        System.out.println("Housing Authority reviewing application of " + c.name);
        for (House h : city.houses) {
            if (h.allocatedTo == null) {
                h.assignCitizen(c);
                break;
            }
        }
    }
}

public class DUMSSystem {
    public static void main(String[] args) {
        CityAuthoritySystem city = new CityAuthoritySystem();
        city.addHouse(new House(1, "Gulshan", 1200));
        city.addHouse(new House(2, "Banani", 900));
        Citizen rahim = new Citizen(101, "Rahim", 15000, 4, "Old Dhaka");
        city.addCitizen(rahim);
        HousingAuthority ha = new HousingAuthority(1, "Dhaka Housing Authority");
        rahim.applyHousing(ha, city);
        city.monitorHousing();
    }
}
```

```
class CityAuthoritySystem {
    List<Citizen> citizens = new ArrayList<>();
    List<House> houses = new ArrayList<>();

    public void addCitizen(Citizen c) {
    }

    public void addHouse(House h) {
        houses.add(h);
    }

    public void monitorHousing() {
        System.out.println("Monitor
        for (House h : houses) {
            if (h.allocatedTo != nul
                System.out
        }
    }
}
```

Others Module

- ✓ System Integration (High-Level Workflow)
- ✓ User Interface (Web/Mobile App):
  - ➢ Citizens apply for housing, report garbage issues, or submit traffic complaints.
  - ➢ Government officials review data and take action.
- ✓ Database:
  - ➢ Stores applicant details, housing units, garbage zones, and traffic signals.
- ✓ AI/ML Components:
  - ➢ Predictive housing allocation.
  - ➢ Optimal garbage truck routing and Smart traffic light automation.
- ✓ APIs for External Systems:
  - ➢ Dhaka City Corporation (garbage trucks).
  - ➢ BRTA (traffic management).

System Scenario-2: "Agricultural Automation System (AAS)"

- ✓ Agriculture is one of the most important sectors in Bangladesh. Farmers face several challenges:
  - ❖ Inefficient irrigation management
  - ❖ Excessive fertilizer usage
  - ❖ Pest and weed attacks
  - ❖ Labor shortages
  - ❖ Low crop yield prediction accuracy
  - ❖ Delayed harvesting decisions
- ✓ The **Agricultural Automation System (AAS)** is a centralized smart farming platform designed to automate the rice cultivation process using **IoT sensors, AI/ML, drones, and automated machinery.**
- ✓ The **system can be accessed through web and mobile applications by farmers, agricultural officers, field workers, and farm managers.**
- ✓ The system **continuously collects field data from sensors and cameras and automatically controls irrigation, fertilization, pest control, and harvesting operations.**

System Goal

- ✓ Develop a smart farming platform that:
  - ❖ Monitors crop growth
  - ❖ Controls irrigation automatically
  - ❖ Detects diseases and pests
  - ❖ Optimizes fertilizer application
  - ❖ Predicts harvesting time
  - ❖ Increases crop production





AAS Main Modules

1.Field Preparation Module

✓ **Responsibilities:**

- ❖ Register farm land
- ❖ Analyze soil condition
- ❖ Measure pH level
- ❖ Detect moisture level
- ❖ Prepare cultivation schedule

✓ **Input:**

- ❖ Soil sensor data
- ❖ Farmer information

✓ **Output:**

- ✓ Land readiness report

2.Planting Module

✓ **Responsibilities:**

- ❖ Manage seed information
- ❖ Schedule planting
- ❖ Control automatic seed planter

✓ **Input:**

- ❖ Seed type
- ❖ Field information

✓ **Output:**

- ❖ Planting confirmation





AAS Main Modules

3. Irrigation Control Module

✓ **Responsibilities:**

- ❖ Monitor soil moisture
- ❖ Control water pumps
- ❖ Manage irrigation schedules

✓ **Input:**

- ❖ Moisture sensors
- ❖ Weather forecast

✓ **Output:**

- ❖ Automatic irrigation decision

4. Fertilization Module

✓ **Responsibilities:**

- ❖ Monitor nutrient levels
- ❖ Calculate fertilizer requirements
- ❖ Schedule fertilizer application

✓ **Input:**

- ❖ Soil nutrients
- ❖ Crop growth stage

✓ **Output:**

- ❖ Fertilizer recommendation

AAS Main Modules

5. Pest & Weed Control Module

✓ **Responsibilities:**

- ❖ Detect pests using camera images
- ❖ Detect weeds
- ❖ Recommend pesticide usage
- ❖ Control spraying drones

✓ **Input:**

- ❖ Drone images
- ❖ Camera feeds

✓ **Output:**

- ❖ Pest treatment plan

6. Growth Monitoring Module

✓ **Responsibilities:**

- ❖ Monitor plant growth
- ❖ Calculate crop health index
- ❖ Estimate yield

✓ **Input:**

- ❖ Drone images
- ❖ Sensor data

✓ **Output:**

- ❖ Growth report

AAS Main Modules

7. Harvesting Module

✓ **Responsibilities:**

- ❖ Predict harvesting date
- ❖ Control harvesting machines

✓ **Input:**

- ❖ Growth stage
- ❖ Weather condition

✓ **Output:**

- ❖ Harvest schedule

8. Post-Harvest Module

✓ **Responsibilities:**

- ❖ Store harvested rice
- ❖ Manage transportation
- ❖ Track warehouse inventory

✓ **Input:**

- ❖ Harvest quantity

✓ Output:

- ❖ Storage report

AAS Main Modules

9. Central Control System

- ✓ Responsibilities:
- ✓ Coordinate all modules
- ✓ Store data
- ✓ Generate reports
- ✓ Run AI prediction models





AAS Main Modules

Step-1: Understanding the Problem Domain

- ✓ **Main Actors**
  - ❖ **Human Actors**
    1. Farmer
    2. Agricultural Officer
    3. Farm Manager
    4. Drone Operator

  - ❖ **System Actors**
    1. Soil Sensor
    2. Moisture Sensor
    3. Weather API
    4. AI/ML Prediction Engine
    5. Irrigation Pump
    6. Drone Sprayer
    7. Harvesting Machine





Use Case 1:
Smart Irrigation Control

- ✓ **Goal**
  - ❖ Automatically irrigate fields based on soil moisture and weather conditions.
- ✓ **Primary Actor**
  - ❖ Farmer
- ✓ **Supporting Actors**
  - ❖ Moisture Sensor
  - ❖ Weather API
  - ❖ Irrigation Pump
  - ❖ AI Irrigation System
- ✓ **Preconditions**
  - ❖ Sensors are connected.
  - ❖ Irrigation pumps are registered.

Main Scenario

1. Moisture sensors continuously monitor soil.
2. Soil moisture drops below threshold.
3. Sensor sends data to AAS.
4. Weather API provides forecast.
5. AI module analyzes moisture and weather.
6. System decides irrigation is necessary.
7. Pump receives activation command.
8. Water is supplied automatically.
9. Moisture reaches target level.
10. Pump stops.
11. Farmer receives notification.

- ✓ **Alternative Scenario**
  - ❖ If rain is expected within a few hours, irrigation is postponed.
- ✓ **Output**
  - ❖ Efficient water management.

Use Case 2
Pest Detection and Control

Goal
Automatically detect and control pest attacks.
Primary Actor
Agricultural Officer
Supporting Actors
Drone Camera
AI Vision System
Drone Sprayer
Preconditions
Drone system operational.

Main Scenario
Drone captures field images.
Images uploaded to AAS.
AI detects pest infection.
Infection severity calculated.
Treatment recommendation generated.
Agricultural Officer approves.
Sprayer drone receives task.
Pesticide sprayed.
Treatment report stored.
Farmer notified.

Alternative Scenario
If no pest is detected, monitoring continues.
Output
Reduced crop loss.

Use Case 3
Harvest Prediction and Scheduling

Goal
Predict optimal harvesting time.
Primary Actor
Farm Manager
Supporting Actors
Growth Monitoring Module
Weather API
Harvesting Machine
Preconditions
Crop growth data available.

Main Scenario
Growth module analyzes crop maturity.
Weather forecast retrieved.
AI predicts harvesting date.
Harvest schedule generated.
Manager reviews schedule.
Harvesting machine assigned.
Harvest operation completed.
Storage information updated.

Alternative Scenario
If severe weather is expected, harvesting date is adjusted.
Output
Timely harvesting and reduced crop damage.

Integrated Scenario:
A Day in Smart Farming Managed by AAS

- ✓ In the morning, soil moisture sensors detect that water levels are below the required threshold. The Irrigation Control Module consults weather forecasts and automatically activates irrigation pumps.
- ✓ Later, drones fly over the rice field and capture images. The Pest & Weed Control Module analyzes the images and detects a pest outbreak in one section of the farm. The Agricultural Officer approves the treatment recommendation, and a spraying drone applies pesticide automatically.
- ✓ In the afternoon, the Growth Monitoring Module evaluates crop maturity and predicts that harvesting should begin within five days. The Harvesting Module schedules harvesting machinery and updates the farmer.
- ✓ Throughout the day, the Central Control System collects data from all modules and provides real-time reports through the farmer's mobile application.
- ✓ The AAS enables data-driven, automated, and efficient farming operations while improving crop yield and reducing labor costs.





OOAD – Main Classes (Domain Model)

- ✓ Farmer
- ✓ AgriculturalOfficer
- ✓ FarmManager

- ✓ Field
- ✓ Crop
- ✓ Sensor
- ✓ SoilSensor
- ✓ MoistureSensor

- ✓ IrrigationController
- ✓ FertilizerController
- ✓ PestController
- ✓ GrowthMonitor
- ✓ HarvestManager

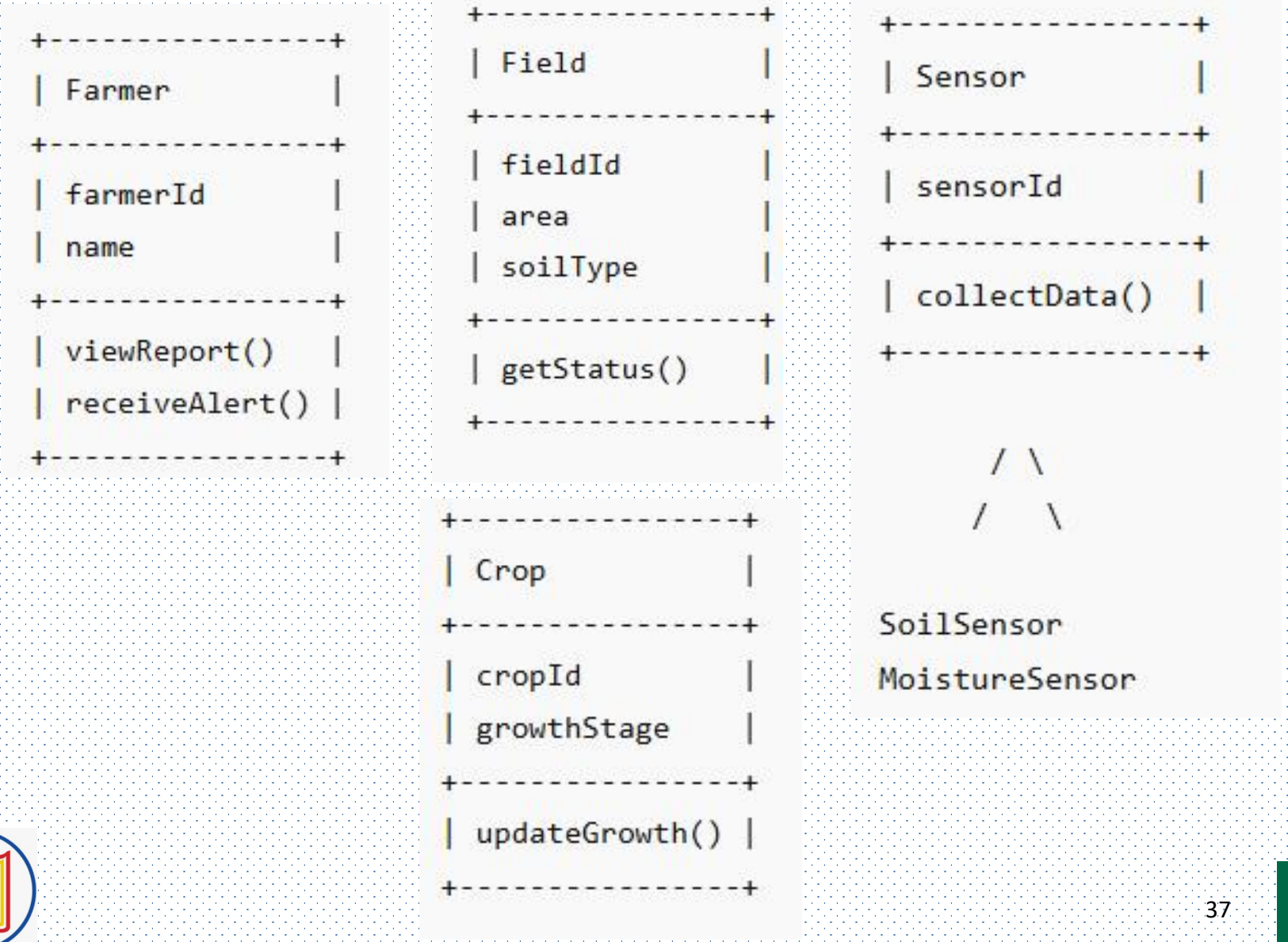
- ✓ Drone
- ✓ SprayerDrone
- ✓ HarvestMachine

- ✓ WeatherService
- ✓ AIEngine
- ✓ ReportGenerator





OOAD – Main Classes (Domain Model)



Suitable Design Patterns for AAS

Module	Design Pattern
Central Control System	**Singleton**
Sensor Notification	**Observer**
AI Decision Engine	**Strategy**
Report Generation	**Factory**
Irrigation Controller	**Command**
Drone Control	**Adapter**
Module Integration	**Facade**





Example UML Sequence Diagram (Irrigation)

- ✓ Farmer
- ✓ |
- ✓ MoistureSensor
- ✓ |
- ✓ AIEngine
- ✓ |
- ✓ IrrigationController
- ✓ |
- ✓ WaterPump
- ✓ Sensor -> AIEngine : sendData()
- ✓ AIEngine -> Controller : irrigate()
- ✓ Controller -> Pump : start()
- ✓ Pump -> Controller : completed()
- ✓ Controller -> Farmer : notify()





Simple Python Demonstration (Observer Pattern)

```
class Farmer:
    def update(self, message):
        print("Farmer Notification:", message)
class MoistureSensor:
    def __init__(self):
        self.observers = []
    def attach(self, observer):
        self.observers.append(observer)
    def notify(self, message):
        for observer in self.observers:
            observer.update(message)
    def check_moisture(self, level):
        if level < 30:
            self.notify(
                "Soil moisture is low. Irrigation started."          )
```

- ✓ farmer = Farmer()
- ✓ sensor = MoistureSensor()
- ✓ sensor.attach(farmer)
- ✓ sensor.check_moisture(25)

- ✓ Output:
  - ❖ Farmer Notification:
    Soil moisture is low. Irrigation started.





Real world object and Benifit

- ✓ The benefit of representing real-world entities—such as Farmer, SensorModule, and IrrigationController—in the Automated Agricultural System as objects in your code.
- ✓ Modeling **Farmer**, **SensorModule**, and **IrrigationController** as objects helps create a clear, modular structure. Each object manages its own functions—such as monitoring soil data or controlling irrigation—making the system easier to update, automate, and integrate. This improves reliability, simplifies troubleshooting, and supports smart, data-driven farming.





Why "Good software design begins before coding"?

- ✓ **Key Concept:** Design is a blueprint; coding is implementation.
- ✓ If the design is weak, the software will be hard to understand, extend, or maintain.
- ✓ A good design ensures:
  - ➢ Clear structure
  - ➢ Easier debugging and testing
  - ➢ Flexibility for future changes
- ✓ Before writing code for housing allocation, we:
  - ➢ Identified core problems (overpopulation, garbage, traffic).
  - ➢ Defined requirements (e.g., "Applicants must submit income proof").
  - ➢ Designed classes (Applicant, HousingUnit) and workflows.





Establishing Initial Software Requirements

- ✓ **Key Concept:** Gather what the system must do (functional) and should do (non-functional).
- ✓ DUMS Example:
- ✓ **Functional Requirements:**
  - ✓ Citizens can apply for housing online.
  - ✓ Garbage trucks are assigned based on zone complaints.
- ✓ **Non-Functional Requirements:**
  - ✓ Scalable to 10M users (Dhaka's population).
  - ✓ Real-time traffic signal updates (<1 sec latency).





OOAD Main Activities : 1. Conceptual Design

- ✓ Focuses on what the system should do
- ✓ **Key Concept:** High-level architecture without technical details.
- ✓ Involves understanding the problem key objects, classes, and their relationships.
- ➢ DUMS Example:
- ✓ **Entities/Objects Identified:**
  - ➢ Applicant, HousingUnit, GarbageZone, TrafficSignal.
- ✓ **Relationship:**
  - ➢ A Citizen submits a Complaint linked to a GarbageZone.
  - ➢ Housing Allocation System matches Applicant to HousingUnit.

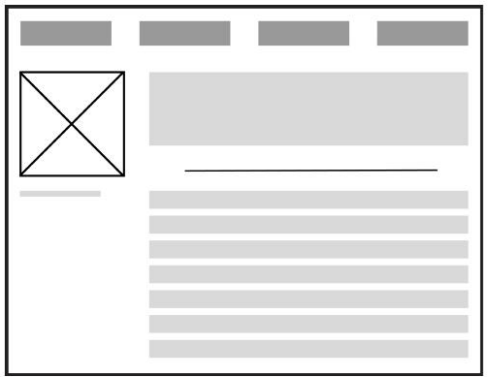




OOAD Main Activities : 1. Conceptual Design

"What things exist in the system, and how are they related?"

- ✓ The conceptual design recognizes appropriate **components, connections, and responsibilities** of the software product.
- ✓ Conceptual designs are expressed or communicated through **conceptual mock-ups**.
- ✓ A visual notations that provide initial thoughts for how requirements will be satisfied.
- ✓ Mock-ups for software involving user interfaces are often presented as **wireframes**, which are a kind of blueprint or basic visual representation of the product.

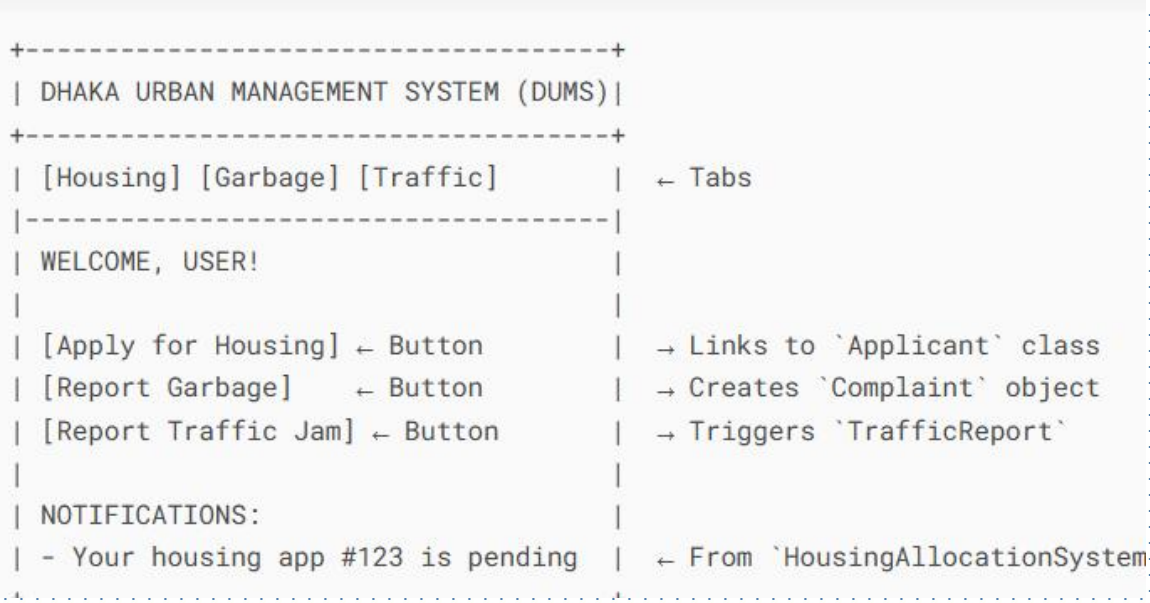






OOAD Main Activities : 1. Conceptual Design

- ✓ Mock-ups show **major components and their relations.**
- ✓ They help identify missing or problematic parts, requiring clarification or further conceptual design.
- ✓ Each component has **specific responsibilities**.
- ✓ Mock-ups focus on structure. not technical details.



OOAD Main Activities : 1. Conceptual Design

Purpose and **Role of Conceptual Mock-ups**

- ✓ **Visualize OO Concepts**
  - ✓ *Example*: UML class diagrams for an Order-Product relationship.
- ✓ **Validate Design Early**
  - ✓ *Example*: Wireframes showing how Cart and Checkout classes interact.
- ✓ **Bridge Client-Developer Gaps**
  - ✓ *Example*: A mock-up of a Dashboard class with widgets (client thinks "features", dev thinks "composition").
- ✓ **Tools**:
  - ➢ UML Diagrams (Class, Sequence)
  - ➢ Wireframing tools (Figma, Balsamiq)





Conceptual Mock-ups illustrate in the Software Design process

- ✓ Conceptual mock-ups are **visual representations** of how the system will look and behave before implementation. They illustrate:

1.User Interface (UI) Flow – how users navigate screens.

2.System Behavior – expected actions when users interact with components.

3.Requirements Validation – clients can confirm if the design matches their needs.

4.Communication Tool – bridges the gap between designers, developers, and stakeholders.

- ✓ **For example, a** conceptual mock-up of a **banking app** may show:
  - ➢ Login screen → Dashboard → Transfer Money form → Confirmation screen.
  - ➢ This allows the client to approve the design before coding begins.
- ✓ Conceptual mock-ups illustrate the **look-and-feel, functionality, and usability** of the software, reducing misunderstandings between clients and developers.





OOAD Main Activities : 2.Technical Design

Key Concept: Low-level implementation plans (algorithms, data structures).

- ✓ Builds on conceptual design to define the technical details of the solution.
- ✓ Conceptual design = outlines major components, connections, and responsibilities.
- ✓ Technical design = explains how responsibilities are implemented.
- ✓ Components are refined into smaller, detailed parts until each has clear technical specifications.
- ✓ Technical diagrams are used to:
  - ➢ Visualize structure and behavior of components.
  - ➢ Capture details not shown in conceptual mock-ups.
- ✓ Example:
  - ➢ Kitchen = a component of a house.
  - ➢ Technical design refines it further (e.g., flooring material must be easy to clean if cooking is frequent).





OOAD Main Activities : 2.Technical Design

Key Concept: Low-level implementation plans (algorithms, data structures).

DUMS Example:

✓ **Algorithms:**

- Housing allocation uses a priority queue (income-based ranking).
- Traffic control uses Dijkstra's algorithm for emergency vehicle routing.

✓ **Database Schema:**

- Applicants table with fields: income, family_size.





OOAD Main Activities : 2.Technical Design

Aspect	Conceptual Design	Technical Design
Purpose	Defines *what* the system should do (major components and their responsibilities).	Defines *how* the system will work (detailed implementation of each component).
Detail Level	High-level overview, not very specific.	Low-level detail, refined until every part is well-defined.
Focus	Identifies major components and their interactions.	Specifies internal structure, materials, methods, and technical details.
Tools	Mock-ups, conceptual diagrams.	Technical diagrams (UML, architecture diagrams, flowcharts, etc.).
House Example	Kitchen is a component of the house.	Kitchen requires flooring; flooring must be easy to clean for frequent cooking use.
Software Example	User Management Module exists in the system.	User Management Module requires login with encryption, password reset via email.

Compromise in Requirements and Design

- ✓ **Compromises may be needed** to create an acceptable solution.
- ✓ **Communication and feedback** with the client are essential.
- ✓ Example: Client wants an open kitchen, but structural support (post/beam)
- ✓ may require adjustment → compromise needed.
- ✓ **Designs may need rework** if:
  - ➢ Conceptual design cannot be achieved in technical design.
  - ➢ Requirements are not fully met.
- ✓ **Early validation is critical**:
  - ➢ Easier to revise during planning than after coding starts.
- ✓ **Larger systems** = more design time, more components, more refinements.
- ✓ **Final agreed design** → technical diagrams form the blueprint for construction/implementation.





Design for Quality Attributes

- ✓ **Broader view in design**: Designing a software system so that it not only performs its required functions but also achieves important **non-functional requirements (quality attributes)** such as performance, convenience, security, reliability, usability, and maintainabilit
- ✓ **Trade-offs are common:**
  - ➢ **Functional Requirements define *what* the system should do**.
  - ➢ **Quality attributes** define *how well* it should work.
- ✓ **Key quality attributes**: Making architectural and design decisions that help the software achieve these desired qualities
- ✓ **Competing attributes**: improving one may weaken another.
- ✓ **Software architects' role**:
  - ➢ Continuously balance attributes.
  - ➢ Decide which attribute is more critical in a given context.
- ✓ **Deadlines matter**: time constraints affect feasibility of quality improvements.
- ✓ **Example (House Door)**:
  - ➢ Too many locks = high security but low convenience.
  - ➢ Good design balances security, convenience, and performance.





Example: Online Shopping syste

Functional Requirements

The system should allow users to:

- Register
- Log in
- Search products
- Add items to the cart
- Pay online
- These describe **what** the system does.

Quality Attribute	Design Decision
Performance	Cache product pages to reduce load time
Security	Encrypt passwords and use HTTPS
Reliability	Maintain database backups and redundant servers
Usability	Provide a simple and intuitive interface
Maintainability	Organize the application into separate modules
Scalability	Use cloud infrastructure to support more users





Example: ATM System

Functional Requirements

- Withdraw cash
- Deposit money
- Check balance

Quality Attribute	Design Decision
Security	PIN verification and encrypted communication
Performance	Complete transactions within a few seconds
Reliability	Ensure operation even during network interruptions
Availability	Keep ATMs operational 24/7
Maintainability	Allow software updates without replacing the hardware





Context and Consequences in Design

- ✓ **Context matters**:
  - ✓ Different environments = different requirements.
  - ✓ Example: Public-facing system needs stronger security than internal corporate system.
- ✓ **Consequences of design choices**:
  - ✓ Some decisions may cause problems later.
  - ✓ Example: Solution works for small data but fails with large data.
- ✓ **Good practices**:
  - ✓ Seek feedback from other developers/stakeholders (design reviews).
  - ✓ Prototype alternative ideas to compare approaches.
  - ✓ Test thoroughly before full implementation.
  - ✓ Use tests with different scenarios (e.g., small vs large data) to find limitations.





Satisfying Qualities in Software Design

- ✓ **Qualities come from requirements**:
  - **Functional requirements** = what the system should do.
  - **Non-functional requirements** = how well the system should do it.
- ✓ **Functional requirements**:
  - Ensure **correctness**.
  - Example: Music app must download and play songs.
- ✓ **Non-functional requirements**:
  - Focus on performance, efficiency, resource usage, maintainability, flexibility, reusability.
  - Example: Music app can only download songs within a memory limit.
- ✓ **Requirements are often incomplete** → clarified through client/end-user interactions.
- ✓ **Trade-offs and compromises** may be needed due to constraints.
  - Example: All cars provide transportation (functional), but differences in acceleration, fuel economy, and handling (non-functional) distinguish minivan vs. sports car.
- ✓ **Verification methods**:
  - Reviews and tests check if qualities are satisfied.
  - End-user feedback validates real-world qualities.

Compromise in Software Design

- ✓ **Multiple perspectives matter**:
  - ✓ Users care about performance, usability.
  - ✓ Developers care about reusability, maintainability.
- ✓ **Software structure affects both sides**.
- ✓ **Common trade-offs**:
  - ➢ **Performance vs. Maintainability**
    - ✓ High-performance code may be harder to understand/maintain.
    - ✓ Backward compatibility may reduce both performance and maintainability.
  - ➢ **Performance vs. Security**
    - ✓ Stronger security adds overhead, lowering performance.
- ✓ **Balance qualities**:
  - ✓ Prioritize what is most important for the system.
  - ✓ Ask: *Can one quality be reduced slightly to improve another?*

Importance of Design and Object-Oriented Thinking

Key Concept: OOAD promotes modularity, reusability, and maintainability.

- **Encapsulation:** Group data and behavior into a single unit (class).
- **Abstraction:** Focus only on essential features, ignoring details.
- **Inheritance:** Reuse common features by extending classes.
- **Polymorphism:** Same action behaves differently depending on the object.

These principles make the design more reusable, flexible, and easier to maintain.





Importance of Design and Object-Oriented Thinking

- ✓ DUMS Example:
  - ➢ **Modularity:** Each module (housing, garbage, traffic) is independent.
  - ➢ **Reusability:** User class inherited by Citizen and Government Official.
  - ➢ **Maintainability:** Changing garbage collection logic doesn't break traffic module.



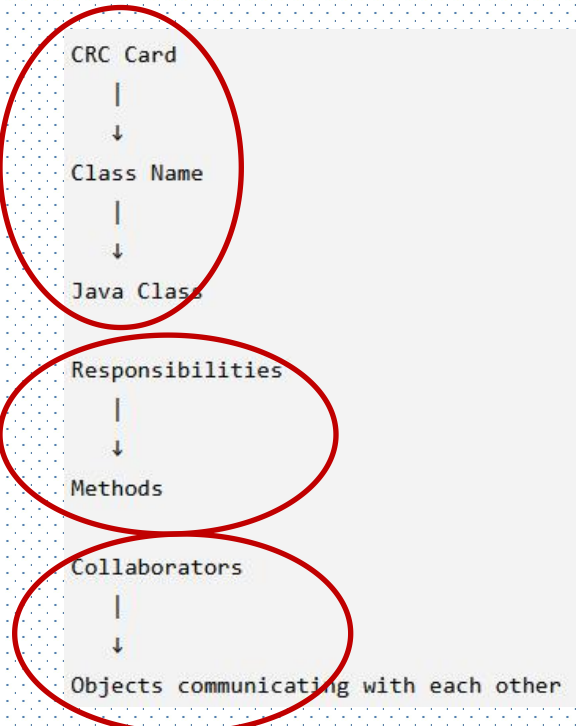


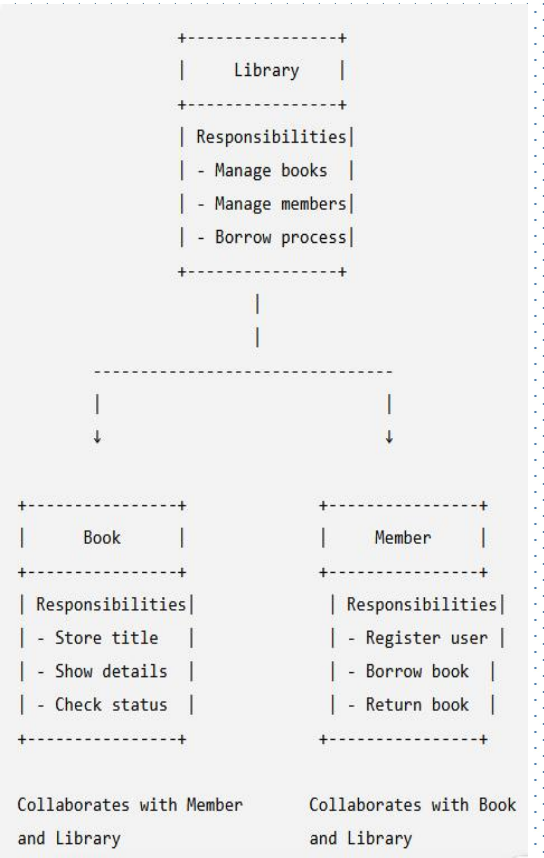
Class Responsibility Collaborator (CRC) Cards

- ✓ **Purpose**:
  - Represent **components, responsibilities, and connections** at a high level.
  - A brainstorming tool for object-oriented design, help refine conceptual design.
- ✓ **Benefits**:
  - Organize system components into **classes**.
  - Identify **responsibilities** of each class.
  - Define how classes **collaborate** with others.
  - Simple, visual tool to support brainstorming and design.
- ✓ **Structure of a CRC Card**:
  - **Top** → Class name.
  - **Left side** → Responsibilities (tasks of the class).
  - **Right side** → Collaborators (other classes it interacts with).
- ✓ **Analogy**:
  - Like index cards used for organizing talking points.
  - Each card = one class in the system.
- ✓ **Each card represents a class:**
  - **Class Name (e.g., Book)**
  - **Responsibilities (what it does, e.g., "Store title, author")**
  - Collaborators (other classes it interacts with, e.g., Member)

Class Responsibility Collaborator (CRC) Cards

Class	Responsibilities	Collaborators
Book	- Store book information- Display book details- Check availability	Member, Library
Member	- Register member- Borrow book- Return book	Book, Library
Library	- Manage books- Manage members- Process borrowing	Book, Member





CRC (Class–Responsibility–Collaborator) Cards

A simple, powerful tool for high-level object-oriented design.

Purpose

- Represent components, responsibilities, and connections at a high level.
- A brainstorming tool for object-oriented design, help refine conceptual design.

Benefits

- Organize system components into classes.
- Identify responsibilities of each class.
- Define how classes collaborate with others.
- Simple, visual tool to support brainstorming and design.

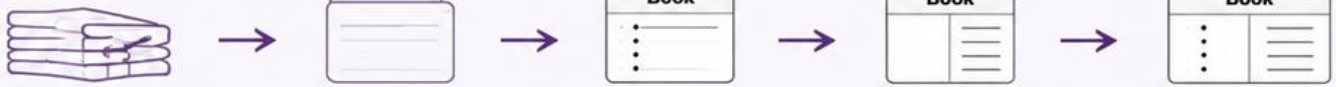
Structure of a CRC Card

Top → Class name.

Left side → Responsibilities (tasks of thie class).

Right side → Collaborators (other classes it interacts with).

Analogy



Like index cards used for organizing talking points.

Each card = one class in the system.

Left side = Responsibilities (what it does)

Right side = Collaborators (who it works with)

Together = complete high-level view of the class.

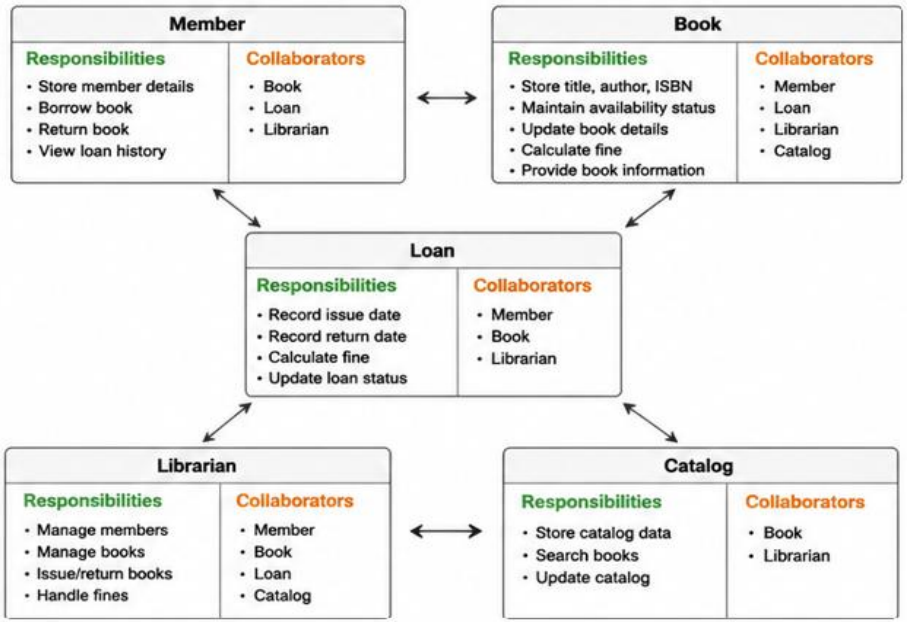
Example: One CRC Card

Book	
Responsibilities	**Collaborators**
• Store title, author, ISBN	• Member
• Maintain availability status	• Loan
• Update book details	• Librarian
• Calculate fine (if returned late)	• Catalog
• Provide book information	

What the class does

Other classes it interacts with

Example: CRC Cards for a Library System



Key Takeaway

CRC cards help you think about classes, where each class is responsible for, and which other classes it needs to work with—before writing any code.

CRC (Class–Responsibility–Collaborator) Cards

A simple, visual tool for high-level object-oriented design.

Purpose

- Represent components, responsibilities, and connections at a high level.
- A brainstorming tool for object-oriented design, help refine conceptual design.

Benefits

- Organize system components into classes.
- Identify responsibilities of each class.
- Define how classes collaborate with others.
- Simple, visual tool to support brainstorming and design.

Structure of a CRC Card

Top → Class name.

Left side → Responsibilities (tasks of thie class).

Right side → Collaborators (other classes it interacts with).

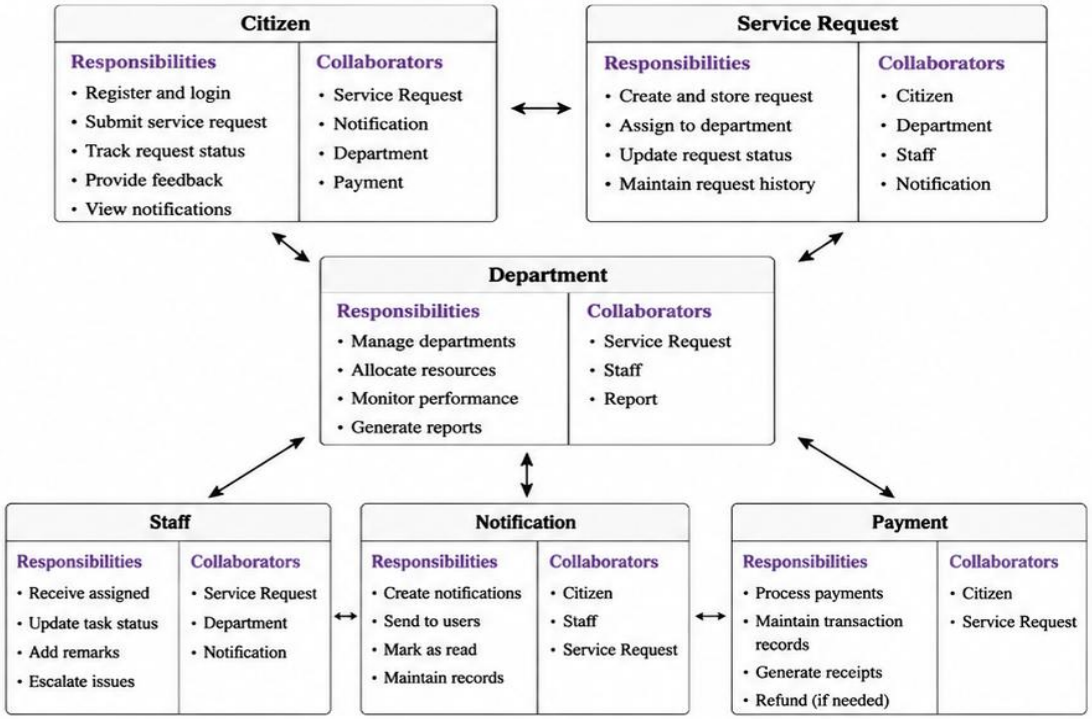
Example: One CRC Card

Citizen	
Responsibilities	**Collaborators**
• Register and login	• Service Request
• Submit service request	• Notification
• Track request status	• Department
• Provide feedback	• Payment
• View notifications	

What the class does

Other classes it interacts with

Example: CRC Cards for Dhaka Urban Management System



Key Takeaway

CRC cards help you think about classes, what each class is responsible for, and which other classes it needs to work with—before writing any code.

System Overview (Dhaka Urban Management System)

Citizens submit requests → Departments assign them to staff → Staff process requests → Notifications and payments are handled → Reports are generated for decision making.

CRC Cards Technique

- ✓ **Key Concept:** A brainstorming tool to design classes.
- ✓ **DUMS Example for Housing AllocationSystem:**

Class	Responsibility	Collaborator
HousingAllocationSystem	Match applicants to housing units	Applicant, HousingUnit
Applicant	Provide personal/financial data	HousingAllocationSystem

- ✓ How It Helps:
  - ✓ Clarifies that Housing Allocation System needs Applicant data to function.
  - ✓ Exposes missing links (e.g., "Should Applicant interact with Garbage Zone? No!").





Scenario: CRC Example

❖ **Problem:** Consider a **Dhaka Urban Management System** scenario: a city resident logs into the smart city portal using their ID and password to verify their identity with the municipal database. The user is then provided options to **report waste collection issues**, **pay utility bills**, **request road maintenance**, or **track service status**. Explain how **CRC (Class–Responsibility–Collaborator)** cards can be utilized to isolate and define the system components, including their attributes, functionalities, and interactions — for example, classes such as *Citizen*, *MunicipalService*, *Complaint*, and *Payment* can be defined along with their responsibilities and collaborating relationships.

✓ **Using CRC Cards in the Dhaka Urban Management System**

CRC (Class–Responsibility–Collaborator) cards help break down the Dhaka Urban Management System into manageable components by defining what each class represents, what it is responsible for, and which other classes it interacts with. This method helps designers isolate key system features and understand how objects collaborate to complete tasks such as logging in, submitting complaints, or paying bills.

For example:

❖ **Citizen Class**

Responsibilities: Authenticate user credentials, manage user profile, initiate service requests.

Collaborators: MunicipalService (to request services), Complaint (to report issues), Payment (to pay bills).





Scenario: CRC Example

❖ MunicipalService Class

Responsibilities: Provide access to available city services such as waste management, road repair, or bill inquiries.

Collaborators: Citizen (service requester), Complaint (for issue reporting), Payment (for bill processing).

❖ Complaint Class

Responsibilities: Record waste collection issues, road maintenance requests, and other public service complaints.

Collaborators: Citizen (complaint initiator), MunicipalService (service handling), ServiceStatus (to track progress).

❖ Payment Class

Responsibilities: Process utility bill payments, verify transaction details, and update user records.

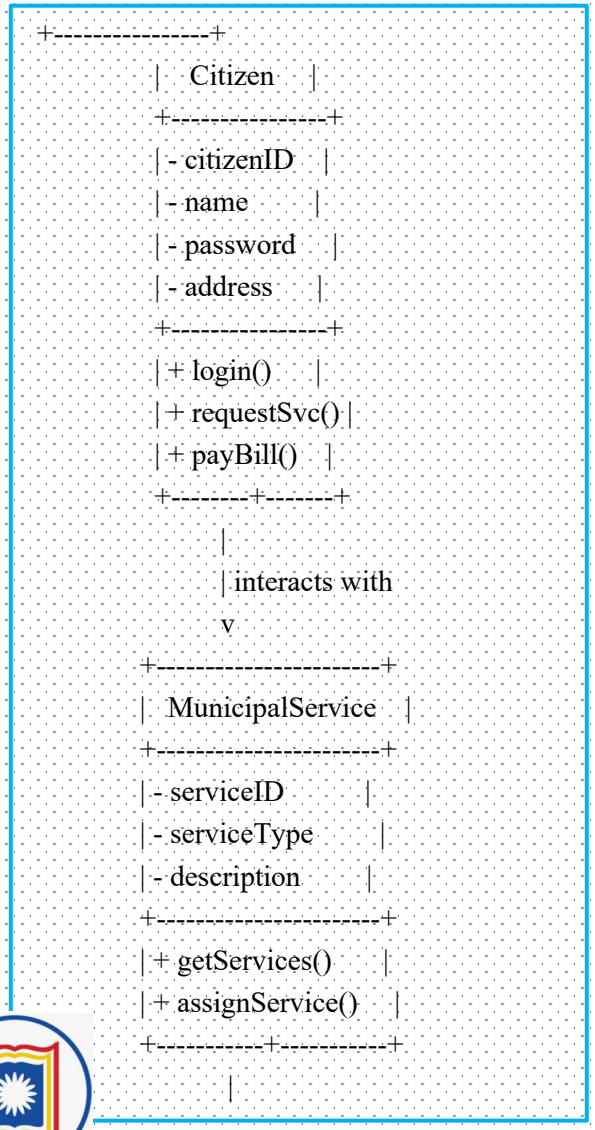
Collaborators: Citizen (payer), MunicipalService (billing information), TransactionGateway (processing).

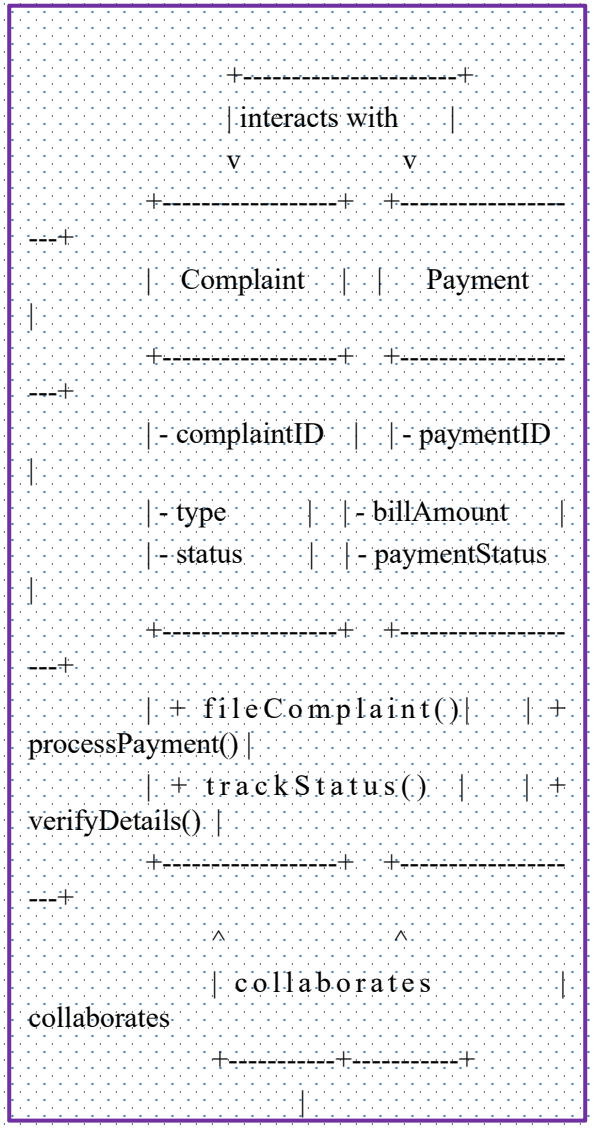
- ✓ Using CRC cards makes it easier to identify each class's attributes (e.g., CitizenID, ServiceType, ComplaintStatus), define their behaviors (e.g., login(), submitComplaint(), processPayment()), and outline how they interact within the system. This approach ensures better system design, promotes modularity, and simplifies implementation and future maintenance.

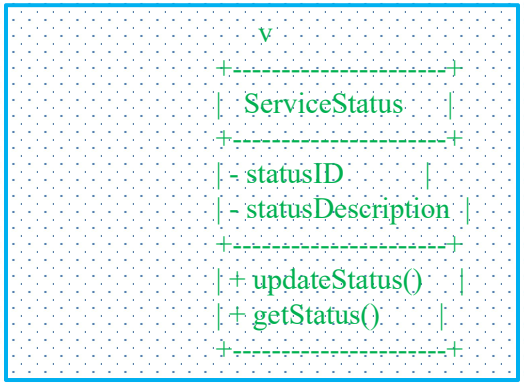




Scenario: CRC Example







- ✓ **Diagram Shows**
- ✓ **Citizen** interacts with:
  - ✓ **MunicipalService** to request services
  - ✓ **Complaint** to report issues
  - ✓ **Payment** to pay bills
- ✓ **MunicipalService** collaborates with:
  - ✓ **Complaint** for issue handling
  - ✓ **Payment** for bill-related operations
  - ✓ **ServiceStatus** to update and trackpro
- ✓ **Complaint** and **Payment** both interact with **ServiceStatus** for tracking results or progress.

Scenario: CRC Example (Automated Agricultural System)

- **Problem:** Consider an Automated Agricultural System scenario: a farmer accesses the automated control system through a mobile app, verifies their identity using a registered farm ID, and is then given options to monitor soil moisture, control irrigation pumps, check crop growth data, or analyze weather forecasts. Explain how CRC (Class–Responsibility–Collaborator) cards can be used to isolate and define the system components, including their attributes, functionalities, and interactions — for instance, classes like Farmer, SensorModule, IrrigationController, and WeatherService can be defined to represent various responsibilities and their collaborations in the automated process.





CRC (Class–Responsibility–Collaborator) Cards

Automated Agriculture System



Scenario: A farmer accesses the automated control system through a mobile app, verifies identity using a registered farm ID, and then gets options to monitor soil moisture, control irrigation pumps, check crop growth data, or analyze weather forecasts.

1. Farmer

Attributes	Responsibilities
• farmID • name • phoneNumber • password	• Login to system • View dashboard • Monitor soil data • Control irrigation • View crop growth • View weather forecast

Collaborators

- MobileApp, AuthService, SensorModule, IrrigationController, CropDataService, WeatherService

2. MobileApp

Attributes	Responsibilities
• appID • version • deviceInfo	• Provide login interface • Show dashboard • Send user requests • Display results/alerts • Manage user session

Collaborators

- Farmer, AuthService, SensorModule, IrrigationController, CropDataService, WeatherService, NotificationService

3. AuthService

Attributes	Responsibilities
• registeredFarmIDs • encryptedData • token	• Verify farm ID • Authenticate user • Generate session token • Manage user access • Logout user

Collaborators

- MobileApp, Farmer

4. SensorModule

Attributes	Responsibilities
• sensorID • location • type • lastReading • soilMoisture	• Collect soil data • Send data to system • Monitor sensor status

Collaborators

- MobileApp, IrrigationController, CropDataService

5. IrrigationController

Attributes	Responsibilities
• pumpID • location • status • mode (Auto/Manual) • lastRunTime	• Receive irrigation command • Control pump (ON/OFF) • Schedule irrigation • Monitor pump status • Send status updates

Collaborators

- MobileApp, SensorService, NotificationService

6. CropDataService

Attributes	Responsibilities
• cropID • cropType • plantingDate • growthStage • healthIndex	• Collect crop growth data • Analyze growth (ON/OFF) • Schedule irrigation • Monitor pump status • Send status updates

Collaborators

- MobileApp, SensorModule

7. WeatherService

Attributes	Responsibilities
• cropID • location • apiKey • lastUpdate	• Collect crop growth data • Forecast weather • Detect crop issues • Send updates

Collaborators

- MobileApp, SensorModuleService

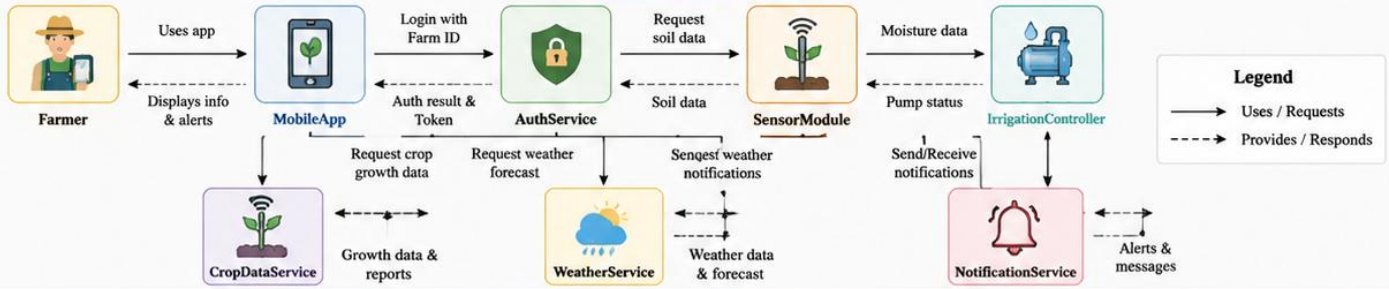
8. NotificationService

Attributes	Responsibilities
• serviceID • channels • messageQueue	• Send alerts • Send reminders • Send weather updates • Send updates

Collaborators

- MobileApp, Farmer, IrrigationController, WeatherService, CropDataService

Class Collaboration Diagram



How the System Works

1. Farmer logs in to the system through the Mobile App using registered Farm ID.
2. AuthService verifies the identity and provides access token.
3. Farmer can view soil moisture (SensorModule), control irrigation pumps (IrrigationController), check crop growth (CropDataService), and analyze weather (WeatherService).
4. NotificationService sends alerts and updates to the farmer.

Key Takeaways

- CRC cards help isolate system components (classes).
- Each class has clear responsibilities (what it does).
- Collaborators show how classes interact with each other.
- This leads to a modular, maintainable and easy-to-extend design.

Is IT possible to fail project work?

- ✓ **Poor Requirements Analysis**
  - If requirements are not captured correctly in the object model (use cases, class diagrams), the system may not meet the client's expectations.
    Example: Missing a key user role in a DUMS leads to incomplete functionality.
  - Building an e-commerce system without understanding payment gateway integration needs.
- ✓ **Incorrect Abstraction and Encapsulations**
  - Using too many low-level details in classes makes the design rigid and hard to extend.
    Example: Putting database logic inside UI classes instead of separating layers. Or Creating a monolithic User class instead of separate Customer, Admin, and Vendor subclasses.
  - Weak Encapsulations: Exposing database connection details in a public class, leading to security risks.
- ✓ **Overuse of Inheritance**
  - Deep inheritance hierarchies make systems complex and fragile.
    Example: A vehicle class hierarchy with too many subclasses becomes unmanageable.



Is IT possible to fail project work?

- ✓ Over-Engineering
  - ➢ Example: Implementing a complex design pattern (e.g., Observer) for a simple notification system.
- ✓ Poor Reusability and Maintainability
- ✓ Mismatch Between Design and Implementation



Trade-offs between the client's need and developer's scope

- ✓ In software design, **trade-offs** occur because clients usually want maximum functionality, low cost, and quick delivery, while developers must work within technical feasibility, resources, and time constraints. Key Trade-offs:
- ✓ **Functionality vs. Cost/Time**
  - ✓ Clients may demand advanced features, but developers must balance feasibility with budget and deadlines.
- ✓ **Performance vs. Flexibility**
  - ✓ Clients may want very high performance, while developers may prefer flexible, maintainable code.
- ✓ **Customization vs. Standardization**
  - ✓ Clients may request customized features, but developers may use standardized frameworks to reduce complexity.



Trade-offs between the client's need and developer's scope

- ✓ In developing an **E-commerce system**,
  - ➢ The **client** wants one-click checkout, real-time delivery tracking, and AI-based recommendations.
  - ➢ The **developer** explains that implementing all at once would increase cost and delay delivery.
  - ➢ **Trade-off:** The system is delivered with checkout and tracking first; AI recommendations are planned for a later release.
- ✓ Trade-offs ensure the project is practical and deliverable while still meeting the essential client requirements.

Client Needs	Developer Constraints	Compromise
"I want all features in 2 months"	Realistic timeline is 6 months	**Prioritize MVP** (Minimum Viable Product)
"The app must work on all devices"	High testing overhead	**Phase-wise rollout** (Mobile-first, then desktop)

Thank You (Q/A)



